Database Management Systems

Summary: Week 1 - 4

Partha Pratim Das

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur
ppd@cse.iitkgp.ac.in

January 26, 2022

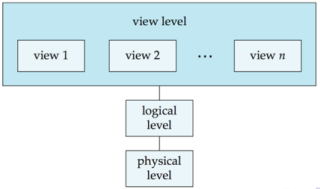
Week 1 Summary

Comparison

Parameter	File Handling via Python	DBMS
Scalability with respect to amount of data	Very difficult to handle insert, update and querying of records	In-built features to provide high scalability for a large number of records
Scalability with respect to changes in structure	Extremely difficult to change the structure of records as in the case of adding or removing attributes	Adding or removing attributes can be done seamlessly using simple SQL queries
Time of execution	In seconds	In milliseconds
Persistence	Data processed using temporary data structures have to be manually updated to the file	Data persistence is ensured via automatic, system induced mechanisms
Robustness	Ensuring robustness of data has to be done manually	Backup, recovery and restore need minimum manual intervention
Security	Difficult to implement in Python (Security at OS level)	User-specific access at database level
Programmer's produc-	Most file access operations involve	Standard and simple built-in queries

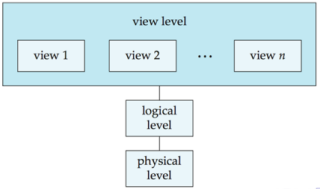
Levels of Abstraction

- **Physical level**: Describes how a record is stored (e.g. blocks of storage)



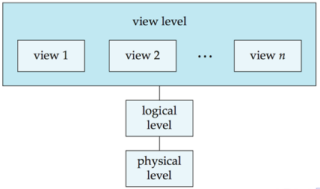
Levels of Abstraction

- **Physical level**: Describes how a record is stored (e.g. blocks of storage)
- **Logical level**: Describes data stored in a database, and the relationships among the data fields (attributes, data types of attributes etc.)



Levels of Abstraction

- **Physical level**: Describes how a record is stored (e.g. blocks of storage)
- **Logical level**: Describes data stored in a database, and the relationships among the data fields (attributes, data types of attributes etc.)
- **View level**: Application programs hide details of data types; it also hides information for security purposes (user interfaces)



Database Engine

- **Storage manager**

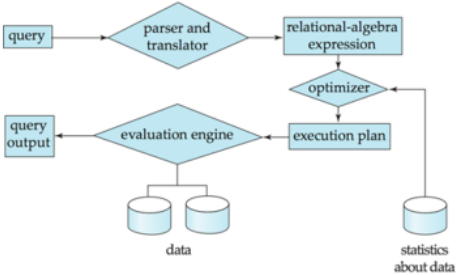


Figure: Query Processing

Database Engine

- **Storage manager**
  - Provides the interface between the low-level data stored in the database and the application programs and queries submitted to the system

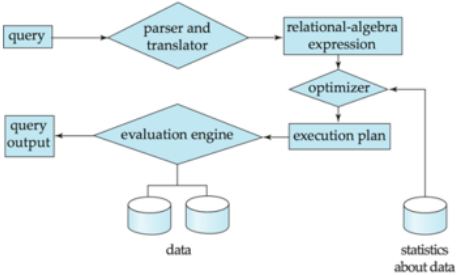


Figure: Query Processing

Database Engine

- **Storage manager**
  - Provides the interface between the low-level data stored in the database and the application programs and queries submitted to the system
  - Interact with the OS file manager

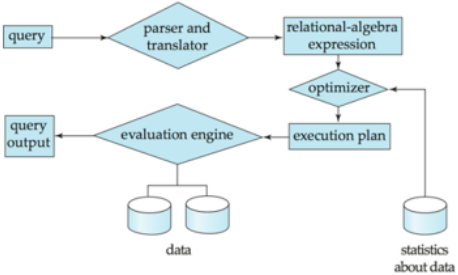


Figure: Query Processing

Database Engine

- **Storage manager**
  - Provides the interface between the low-level data stored in the database and the application programs and queries submitted to the system
  - Interact with the OS file manager
  - Efficient storing, retrieving and updating of data

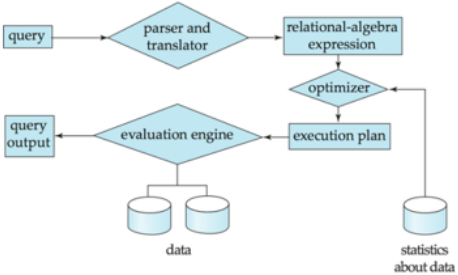


Figure: Query Processing

Database Engine

- **Storage manager**
  - Provides the interface between the low-level data stored in the database and the application programs and queries submitted to the system
  - Interact with the OS file manager
  - Efficient storing, retrieving and updating of data
- **Query Processing**

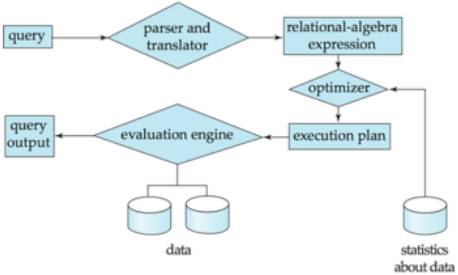


Figure: Query Processing

Database Engine

- **Storage manager**
  - Provides the interface between the low-level data stored in the database and the application programs and queries submitted to the system
  - Interact with the OS file manager
  - Efficient storing, retrieving and updating of data
- **Query Processing**
  - Parsing and translation → Optimization → Evaluation

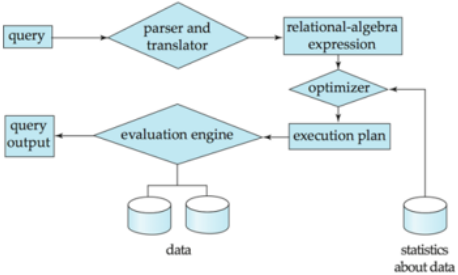


Figure: Query Processing

Database Engine (cont..)

- **Transaction Management**

Database Engine (cont..)

- **Transaction Management**
  - A transaction is a collection of operations that performs a single logical function in a database application

Database Engine (cont..)

- **Transaction Management**
  - A transaction is a collection of operations that performs a single logical function in a database application
  - Transaction-management component - ensures that the database remains in a consistent state despite system failures and transaction failures.

Database Engine (cont..)

- **Transaction Management**
  - A transaction is a collection of operations that performs a single logical function in a database application
  - Transaction-management component - ensures that the database remains in a consistent state despite system failures and transaction failures.
  - Concurrency-control manager - controls the interaction among the concurrent transactions

Week 2 Summary

Module 6 Recap

- **Attribute Types** - The set of allowed values for each attribute is called the domain of the attribute. (i.e. Alphanumeric string, Alpha string, Date, number etc.)

Module 6 Recap

- **Attribute Types** - The set of allowed values for each attribute is called the domain of the attribute. (i.e. Alphanumeric string, Alpha string, Date, number etc.)
- **Schema** - $R = (A_1, A_2, \cdots, A_n)$ is a relation schema where $A_1, A_2, \cdots, A_n$ are attributes.

Module 6 Recap

- **Attribute Types** - The set of allowed values for each attribute is called the domain of the attribute. (i.e. Alphanumeric string, Alpha string, Date, number etc.)
- **Schema** - $R = (A_1, A_2, \cdots, A_n)$ is a relation schema where $A_1, A_2, \cdots, A_n$ are attributes.
- **Instances** : The collection of information stored in the database at a particular moment is called an instance of the database.

Module 6 Recap

- **Attribute Types** - The set of allowed values for each attribute is called the domain of the attribute. (i.e. Alphanumeric string, Alpha string, Date, number etc.)
- **Schema** - $R = (A_1, A_2, \cdots, A_n)$ is a relation schema where $A_1, A_2, \cdots, A_n$ are attributes.
- **Instances** : The collection of information stored in the database at a particular moment is called an instance of the database.
- Let $K \subseteq R$, where R is the set of attributes in the relation.

Module 6 Recap

- **Attribute Types** - The set of allowed values for each attribute is called the domain of the attribute. (i.e. Alphanumeric string, Alpha string, Date, number etc.)
- **Schema** - $R = (A_1, A_2, \cdots, A_n)$ is a relation schema where $A_1, A_2, \cdots, A_n$ are attributes.
- **Instances** : The collection of information stored in the database at a particular moment is called an instance of the database.
- Let $K \subseteq R$, where R is the set of attributes in the relation.
- K is a **super key** of R if values for K are sufficient to identify a unique tuple of each possible relation $r(R)$.

Module 6 Recap

- **Attribute Types** - The set of allowed values for each attribute is called the domain of the attribute. (i.e. Alphanumeric string, Alpha string, Date, number etc.)
- **Schema** - $R = (A_1, A_2, \cdots, A_n)$ is a relation schema where $A_1, A_2, \cdots, A_n$ are attributes.
- **Instances** : The collection of information stored in the database at a particular moment is called an instance of the database.
- Let $K \subseteq R$, where R is the set of attributes in the relation.
- K is a **super key** of R if values for K are sufficient to identify a unique tuple of each possible relation $r(R)$.
- Super key K is a **candidate key** if K is minimal.

Module 6 Recap

- **Attribute Types** - The set of allowed values for each attribute is called the domain of the attribute. (i.e. Alphanumeric string, Alpha string, Date, number etc.)
- **Schema** - $R = (A_1, A_2, \cdots, A_n)$ is a relation schema where $A_1, A_2, \cdots, A_n$ are attributes.
- **Instances** : The collection of information stored in the database at a particular moment is called an instance of the database.
- Let $K \subseteq R$, where R is the set of attributes in the relation.
- K is a **super key** of R if values for K are sufficient to identify a unique tuple of each possible relation $r(R)$.
- Super key K is a **candidate key** if K is minimal.
- One of the candidate keys is selected to be the **primary key**.

Module 6 Recap

- **Attribute Types** - The set of allowed values for each attribute is called the domain of the attribute. (i.e. Alphanumeric string, Alpha string, Date, number etc.)
- **Schema** - $R = (A_1, A_2, \cdots, A_n)$ is a relation schema where $A_1, A_2, \cdots, A_n$ are attributes.
- **Instances** : The collection of information stored in the database at a particular moment is called an instance of the database.
- Let $K \subseteq R$, where R is the set of attributes in the relation.
- K is a **super key** of R if values for K are sufficient to identify a unique tuple of each possible relation $r(R)$.
- Super key K is a **candidate key** if K is minimal.
- One of the candidate keys is selected to be the **primary key**.
- A **surrogate key** (or synthetic key) in a database is a unique identifier for either an entity in the modeled world or an object in the database.

Module 6 Recap (Cont.)

- **Secondary / Alternate Key** : candidate keys other than primary key.

Module 6 Recap (Cont.)

- **Secondary / Alternate Key** : candidate keys other than primary key.
- **Simple key** : Consists of a single attribute.

Module 6 Recap (Cont.)

- **Secondary / Alternate Key** : candidate keys other than primary key.
- **Simple key** : Consists of a single attribute.
- **Composite key** : Consists of more than one attribute to uniquely identify each tuples in a relation.

Module 6 Recap (Cont.)

- **Secondary / Alternate Key** : candidate keys other than primary key.
- **Simple key** : Consists of a single attribute.
- **Composite key** : Consists of more than one attribute to uniquely identify each tuples in a relation.
- **Foreign key** : Value in one relation must appear in another.

Module 6 Recap (Cont.)

- **Secondary / Alternate Key** : candidate keys other than primary key.
- **Simple key** : Consists of a single attribute.
- **Composite key** : Consists of more than one attribute to uniquely identify each tuples in a relation.
- **Foreign key** : Value in one relation must appear in another.
- **Compound key** : consists of more than one attribute to uniquely identify an entity occurrence.

Module 6 Recap (Cont.)

- **Secondary / Alternate Key** : candidate keys other than primary key.
- **Simple key** : Consists of a single attribute.
- **Composite key** : Consists of more than one attribute to uniquely identify each tuples in a relation.
- **Foreign key** : Value in one relation must appear in another.
- **Compound key** : consists of more than one attribute to uniquely identify an entity occurrence.
- **Relational Query language** :

Module 6 Recap (Cont.)

- **Secondary / Alternate Key** : candidate keys other than primary key.
- **Simple key** : Consists of a single attribute.
- **Composite key** : Consists of more than one attribute to uniquely identify each tuples in a relation.
- **Foreign key** : Value in one relation must appear in another.
- **Compound key** : consists of more than one attribute to uniquely identify an entity occurrence.
- **Relational Query language** :
  - **Procedural programming** : requires that the programmer tell the computer what to do. That is, how to get the output for the range of required inputs.

Module 6 Recap (Cont.)

- **Secondary / Alternate Key** : candidate keys other than primary key.
- **Simple key** : Consists of a single attribute.
- **Composite key** : Consists of more than one attribute to uniquely identify each tuples in a relation.
- **Foreign key** : Value in one relation must appear in another.
- **Compound key** : consists of more than one attribute to uniquely identify an entity occurrence.
- **Relational Query language** :
  - **Procedural programming** : requires that the programmer tell the computer what to do. That is, how to get the output for the range of required inputs.
  - **Declarative programming** : requires a more descriptive style. The programmer must know what relationships hold between various entities

Module 7 Recap

- **select operation** : selection of rows(tuples). i.e: $\sigma_{D>5}(r)$

Module 7 Recap

- **select operation** : selection of rows(tuples). i.e: $\sigma_{D>5}(r)$
- **Project operation** : selection of columns (Attributes). i.e. $\pi_{A,C}(r)$

Module 7 Recap

- **select operation** : selection of rows(tuples). i.e: $\sigma_{D>5}(r)$
- **Project operation** : selection of columns (Attributes). i.e. $\pi_{A,C}(r)$
- **Union** : union of two relations. i.e. $r \cup s$

Module 7 Recap

- **select operation** : selection of rows(tuples). i.e: $\sigma_{D>5}(r)$
- **Project operation** : selection of columns (Attributes). i.e. $\pi_{A,C}(r)$
- **Union** : union of two relations. i.e. $r \cup s$
- **Difference** : Set difference of two relations. i.e. $r - s$

Module 7 Recap

- **select operation** : selection of rows(tuples). i.e: $\sigma_{D>5}(r)$
- **Project operation** : selection of columns (Attributes). i.e. $\pi_{A,C}(r)$
- **Union** : union of two relations. i.e. $r \cup s$
- **Difference** : Set difference of two relations. i.e. $r - s$
- **Intersection** : Set intersection of two relations. i.e. $r \cap s$

Module 7 Recap

- **select operation** : selection of rows(tuples). i.e: $\sigma_{D>5}(r)$
- **Project operation** : selection of columns (Attributes). i.e. $\pi_{A,C}(r)$
- **Union** : union of two relations. i.e. $r \cup s$
- **Difference** : Set difference of two relations. i.e. $r - s$
- **Intersection** : Set intersection of two relations. i.e. $r \cap s$
- **Cartesian Product** : Joining two relations - Cartesian-product. i.e. $r \times s$

Module 7 Recap

- **select operation** : selection of rows(tuples). i.e: $\sigma_{D>5}(r)$
- **Project operation** : selection of columns (Attributes). i.e. $\pi_{A,C}(r)$
- **Union** : union of two relations. i.e. $r \cup s$
- **Difference** : Set difference of two relations. i.e. $r - s$
- **Intersection** : Set intersection of two relations. i.e. $r \cap s$
- **Cartesian Product** : Joining two relations - Cartesian-product. i.e. $r \times s$
- **Natural Join** : natural join operation on two relations matches tuples whose values are the same on all attribute names that are common to both relations. i.e. $r \bowtie s$

Module 8 Recap

- History of SQL

Module 8 Recap

- History of SQL
- **Data Definition Language (DDL)** : The SQL DDL provides commands for defining relation schemas, deleting relations, and modifying relation schema. (i.e. `CREATE TABLE`, `DROP TABLE`, `ALTER` etc.)

Module 8 Recap

- History of SQL
- **Data Definition Language (DDL)** : The SQL DDL provides commands for defining relation schemas, deleting relations, and modifying relation schema. (i.e. `CREATE TABLE`, `DROP TABLE`, `ALTER` etc.)
- **Data Manipulation Language (DML)** : The SQL DML provides the ability to query information from the database and to insert tuples into, delete tuples from, and modify tuples in the database. (i.e. `UPDATE`, `INSERT`, `DELETE` etc.)

Module 8 Recap

- History of SQL
- **Data Definition Language (DDL)** : The SQL DDL provides commands for defining relation schemas, deleting relations, and modifying relation schema. (i.e. `CREATE TABLE`, `DROP TABLE`, `ALTER` etc.)
- **Data Manipulation Language (DML)** : The SQL DML provides the ability to query information from the database and to insert tuples into, delete tuples from, and modify tuples in the database. (i.e. `UPDATE`, `INSERT`, `DELETE` etc.)
- **Basic SQL structure**: `SELECT` $A_1, A_2, \ldots, A_n$ `FROM` $r_1, r_2, \ldots, r_m$ `WHERE` P

Module 8 Recap

- History of SQL
- **Data Definition Language (DDL)** : The SQL DDL provides commands for defining relation schemas, deleting relations, and modifying relation schema. (i.e. `CREATE TABLE`, `DROP TABLE`, `ALTER` etc.)
- **Data Manipulation Language (DML)** : The SQL DML provides the ability to query information from the database and to insert tuples into, delete tuples from, and modify tuples in the database. (i.e. `UPDATE`, `INSERT`, `DELETE` etc.)
- **Basic SQL structure**: `SELECT` $A_1, A_2, \ldots, A_n$ `FROM` $r_1, r_2, \ldots, r_m$ `WHERE` P
  - **Select clause**: lists the attributes desired in the result of a query.

Module 8 Recap

- History of SQL
- **Data Definition Language (DDL)** : The SQL DDL provides commands for defining relation schemas, deleting relations, and modifying relation schema. (i.e. `CREATE TABLE`, `DROP TABLE`, `ALTER` etc.)
- **Data Manipulation Language (DML)** : The SQL DML provides the ability to query information from the database and to insert tuples into, delete tuples from, and modify tuples in the database. (i.e. `UPDATE`, `INSERT`, `DELETE` etc.)
- **Basic SQL structure**: `SELECT` $A_1, A_2, \ldots, A_n$ `FROM` $r_1, r_2, \ldots, r_m$ `WHERE` P
  - **Select clause**: lists the attributes desired in the result of a query.
  - **Where clause** : specifies conditions that the result must satisfy.

Module 8 Recap

- History of SQL
- **Data Definition Language (DDL)** : The SQL DDL provides commands for defining relation schemas, deleting relations, and modifying relation schema. (i.e. `CREATE TABLE`, `DROP TABLE`, `ALTER` etc.)
- **Data Manipulation Language (DML)** : The SQL DML provides the ability to query information from the database and to insert tuples into, delete tuples from, and modify tuples in the database. (i.e. `UPDATE`, `INSERT`, `DELETE` etc.)
- **Basic SQL structure**: `SELECT` $A_1, A_2, \ldots, A_n$ `FROM` $r_1, r_2, \ldots, r_m$ `WHERE` P
  - **Select clause**: lists the attributes desired in the result of a query.
  - **Where clause** : specifies conditions that the result must satisfy.
  - **From clause** : lists the relations involved in the query.

Module 9 Recap

- **Cartesian Product** : The Cartesian product instructor X teaches. It generates every possible instructor-teaches pair, with all attributes from both relations.

Module 9 Recap

- **Cartesian Product** : The Cartesian product instructor X teaches. It generates every possible instructor-teaches pair, with all attributes from both relations.

SQL query

```
select * from instructor, teaches
```

Module 9 Recap

- **Cartesian Product** : The Cartesian product instructor X teaches. It generates every possible instructor-teaches pair, with all attributes from both relations.

SQL query

```
select * from instructor, teaches
```

- **Rename AS Operation** : The SQL allows renaming relations and attributes using the as clause. (i.e. `old_name as new_name`)

Module 9 Recap

- **Cartesian Product** : The Cartesian product instructor X teaches. It generates every possible instructor-teaches pair, with all attributes from both relations.

SQL query

```
select * from instructor, teaches
```

- **Rename AS Operation** : The SQL allows renaming relations and attributes using the as clause. (i.e. `old_name as new_name`)
- **String Operation** : SQL includes a string-matching operator for comparisons on character strings. The operator `like` uses patterns that are described using two special characters:

Module 9 Recap

- **Cartesian Product** : The Cartesian product instructor X teaches. It generates every possible instructor-teaches pair, with all attributes from both relations.

SQL query

```
select * from instructor, teaches
```

- **Rename AS Operation** : The SQL allows renaming relations and attributes using the as clause. (i.e. `old_name as new_name`)
- **String Operation** : SQL includes a string-matching operator for comparisons on character strings. The operator `like` uses patterns that are described using two special characters:
  - percent (%). The % character matches any substring

Module 9 Recap

- **Cartesian Product** : The Cartesian product instructor X teaches. It generates every possible instructor-teaches pair, with all attributes from both relations.

SQL query

```
select * from instructor, teaches
```

- **Rename AS Operation** : The SQL allows renaming relations and attributes using the as clause. (i.e. `old_name as new_name`)
- **String Operation** : SQL includes a string-matching operator for comparisons on character strings. The operator `like` uses patterns that are described using two special characters:
  - percent (%). The % character matches any substring
  - underscore (_). The _ character matches any single character

Module 9 Recap

- **Cartesian Product** : The Cartesian product instructor X teaches. It generates every possible instructor-teaches pair, with all attributes from both relations.

SQL query

```
select * from instructor, teaches
```

- **Rename AS Operation** : The SQL allows renaming relations and attributes using the as clause. (i.e. `old_name as new_name`)
- **String Operation** : SQL includes a string-matching operator for comparisons on character strings. The operator `like` uses patterns that are described using two special characters:
  - percent (%). The % character matches any substring
  - underscore (_). The _ character matches any single character
- **Order By** : We may specify `desc` for descending order or `asc` for ascending order, for each attribute; ascending order is the default. (i.e. `order by name desc`)

Module 9 Recap (Cont.)

- **Select top/ Fetch** : The `Select Top` clause is used to specify the number of records to return.

Module 9 Recap (Cont.)

- **Select top/ Fetch** : The `Select Top` clause is used to specify the number of records to return.

SQL Server Example

```
select top 10 distinct name from instructor
```

Oracle uses `fetch first n rows only` and `rownum`

Module 9 Recap (Cont.)

- **Select top/ Fetch** : The `Select Top` clause is used to specify the number of records to return.

SQL Server Example

```
select top 10 distinct name from instructor
```

Oracle uses `fetch first n rows only` and `rownum`

Oracle Example

```
select distinct name from instructor order by name
fetch first 10 rows only
```

Module 9 Recap (Cont.)

- **Select top/ Fetch** : The `Select Top` clause is used to specify the number of records to return.

SQL Server Example

```
select top 10 distinct name from instructor
```

Oracle uses `fetch first n rows only` and `rownum`

Oracle Example

```
select distinct name from instructor order by name
fetch first 10 rows only
```

- **Where Clause Predicate** : SQL includes a `between` comparison operator, Tuple comparison.

Module 9 Recap (Cont.)

- **Select top/ Fetch** : The `Select Top` clause is used to specify the number of records to return.

SQL Server Example

```
select top 10 distinct name from instructor
```

Oracle uses `fetch first n rows only` and `rownum`

Oracle Example

```
select distinct name from instructor order by name
fetch first 10 rows only
```

- **Where Clause Predicate** : SQL includes a `between` comparison operator, Tuple comparison.
- **IN operator** : The `in` operator allows you to specify multiple values in a where clause.

Module 9 Recap (Cont.)

- **Select top/ Fetch** : The `Select Top` clause is used to specify the number of records to return.

SQL Server Example

```
select top 10 distinct name from instructor
```

Oracle uses `fetch first n rows only` and `rownum`

Oracle Example

```
select distinct name from instructor order by name
fetch first 10 rows only
```

- **Where Clause Predicate** : SQL includes a `between` comparison operator, Tuple comparison.
- **IN operator** : The `in` operator allows you to specify multiple values in a where clause.
- **Duplicates** : In relations with duplicates, SQL can define how many copies of tuples appear in the result. Multiset versions of some relational algebra operators.

Module 10 Recap

- **Set Operations** : `union`, `intersect`, `except`

Module 10 Recap

- **Set Operations** : `union`, `intersect`, `except`
  - Each of the above operations automatically eliminates duplicates.

Module 10 Recap

- **Set Operations** : `union`, `intersect`, `except`
  - Each of the above operations automatically eliminates duplicates.
  - To retain all duplicates use the corresponding multiset versions `union all`, `intersect all`, and `except all`.

Module 10 Recap

- **Set Operations** : `union`, `intersect`, `except`
  - Each of the above operations automatically eliminates duplicates.
  - To retain all duplicates use the corresponding multiset versions `union all`, `intersect all`, and `except all`.
- **Null Values** : `null` signifies an unknown value or that a value does not exist.

Module 10 Recap

- **Set Operations** : `union`, `intersect`, `except`
  - Each of the above operations automatically eliminates duplicates.
  - To retain all duplicates use the corresponding multiset versions `union all`, `intersect all`, and `except all`.
- **Null Values** : `null` signifies an unknown value or that a value does not exist.
- **Aggregate Functions** :

Module 10 Recap

- **Set Operations** : `union`, `intersect`, `except`
  - Each of the above operations automatically eliminates duplicates.
  - To retain all duplicates use the corresponding multiset versions `union all`, `intersect all`, and `except all`.
- **Null Values** : `null` signifies an unknown value or that a value does not exist.
- **Aggregate Functions** :
  - `avg`: return average value

Module 10 Recap

- **Set Operations** : `union`, `intersect`, `except`
  - Each of the above operations automatically eliminates duplicates.
  - To retain all duplicates use the corresponding multiset versions `union all`, `intersect all`, and `except all`.
- **Null Values** : `null` signifies an unknown value or that a value does not exist.
- **Aggregate Functions** :
  - `avg`: return average value
  - `min`: return min value

Module 10 Recap

- **Set Operations** : `union`, `intersect`, `except`
  - Each of the above operations automatically eliminates duplicates.
  - To retain all duplicates use the corresponding multiset versions `union all`, `intersect all`, and `except all`.
- **Null Values** : `null` signifies an unknown value or that a value does not exist.
- **Aggregate Functions** :
  - `avg`: return average value
  - `min`: return min value
  - `max`: return max value

Module 10 Recap

- **Set Operations** : `union`, `intersect`, `except`
  - Each of the above operations automatically eliminates duplicates.
  - To retain all duplicates use the corresponding multiset versions `union all`, `intersect all`, and `except all`.
- **Null Values** : `null` signifies an unknown value or that a value does not exist.
- **Aggregate Functions** :
  - `avg`: return average value
  - `min`: return min value
  - `max`: return max value
  - `sum`: return sum of values

Module 10 Recap

- **Set Operations** : `union`, `intersect`, `except`
  - Each of the above operations automatically eliminates duplicates.
  - To retain all duplicates use the corresponding multiset versions `union all`, `intersect all`, and `except all`.
- **Null Values** : `null` signifies an unknown value or that a value does not exist.
- **Aggregate Functions** :
  - `avg`: return average value
  - `min`: return min value
  - `max`: return max value
  - `sum`: return sum of values
  - `count`: return number of values

Module 10 Recap (Cont.)

- **Group By** :

Module 10 Recap (Cont.)

- **Group By** :
  - The attribute or attributes given in the `group by` clause are used to form groups.

Module 10 Recap (Cont.)

- **Group By** :
  - The attribute or attributes given in the `group by` clause are used to form groups.
  - Tuples with the same value on all attributes in the `group by` clause are placed in one group.

Module 10 Recap (Cont.)

- **Group By** :
  - The attribute or attributes given in the `group by` clause are used to form groups.
  - Tuples with the same value on all attributes in the `group by` clause are placed in one group.
- **Having** : predicates in the `having` clause are applied after the formation of groups.

Module 10 Recap (Cont.)

- **Group By** :
  - The attribute or attributes given in the `group by` clause are used to form groups.
  - Tuples with the same value on all attributes in the `group by` clause are placed in one group.
- **Having** : predicates in the `having` clause are applied after the formation of groups.
- **Null Values** : All aggregate operations except `count(*)` ignore tuples with null values on the aggregated attributes.

Week 3 Summary

Module 11

- **Various basic SQL features through example workout**

Module 11

- **Various basic SQL features through example workout**
- `select distinct` - removing the duplicates

Module 11

- **Various basic SQL features through example workout**
- `select distinct` - removing the duplicates
- `select all` - (duplicate retention is the default)

Module 11

- **Various basic SQL features through example workout**
- `select distinct` - removing the duplicates
- `select all` - (duplicate retention is the default)
- Cartesian Product

Module 11

- **Various basic SQL features through example workout**
- `select distinct` - removing the duplicates
- `select all` - (duplicate retention is the default)
- Cartesian Product
- Rename AS Operation

Module 11

- **Various basic SQL features through example workout**
- `select distinct` - removing the duplicates
- `select all` - (duplicate retention is the default)
- Cartesian Product
- Rename AS Operation
- `where` : `AND` and `OR`

Module 11

- **Various basic SQL features through example workout**
- `select distinct` - removing the duplicates
- `select all` - (duplicate retention is the default)
- Cartesian Product
- Rename AS Operation
- `where` : `AND` and `OR`
- String Operations `like` with `%` and `_`

Module 11

- **Various basic SQL features through example workout**
- `select distinct` - removing the duplicates
- `select all` - (duplicate retention is the default)
- Cartesian Product
- Rename AS Operation
- `where` : `AND` and `OR`
- String Operations `like` with `%` and `_`
- `order by` - `ASC` and `DESC`

Module 11

- **Various basic SQL features through example workout**
- `select distinct` - removing the duplicates
- `select all` - (duplicate retention is the default)
- Cartesian Product
- Rename AS Operation
- `where` : `AND` and `OR`
- String Operations `like` with `%` and `_`
- `order by` - `ASC` and `DESC`
- `in` Operator

Module 11

- **Various basic SQL features through example workout**
- `select distinct` - removing the duplicates
- `select all` - (duplicate retention is the default)
- Cartesian Product
- Rename AS Operation
- `where` : `AND` and `OR`
- String Operations `like` with `%` and `_`
- `order by` - `ASC` and `DESC`
- `in` Operator
- **Set Operations**

Module 11

- **Various basic SQL features through example workout**
- `select distinct` - removing the duplicates
- `select all` - (duplicate retention is the default)
- Cartesian Product
- Rename AS Operation
- `where` : `AND` and `OR`
- String Operations `like` with `%` and `_`
- `order by` - `ASC` and `DESC`
- `in` Operator
- **Set Operations**
  - ▷ `union` - union removes all duplicates.

Module 11

- **Various basic SQL features through example workout**
- `select distinct` - removing the duplicates
- `select all` - (duplicate retention is the default)
- Cartesian Product
- Rename AS Operation
- `where` : `AND` and `OR`
- String Operations `like` with `%` and `_`
- `order by` - `ASC` and `DESC`
- `in` Operator
- **Set Operations**
  - `union` - union removes all duplicates.
  - Use `union all` instead of `union` to retain the duplicates.

Module 11

- **Various basic SQL features through example workout**
- `select distinct` - removing the duplicates
- `select all` - (duplicate retention is the default)
- Cartesian Product
- Rename AS Operation
- `where` : `AND` and `OR`
- String Operations `like` with `%` and `_`
- `order by` - `ASC` and `DESC`
- `in` Operator
- **Set Operations**
  - `union` - union removes all duplicates.
  - Use `union all` instead of `union` to retain the duplicates.
  - `intersect`

Module 11

- **Various basic SQL features through example workout**
- `select distinct` - removing the duplicates
- `select all` - (duplicate retention is the default)
- Cartesian Product
- Rename AS Operation
- `where` : `AND` and `OR`
- String Operations `like` with `%` and `_`
- `order by` - `ASC` and `DESC`
- `in` Operator
- **Set Operations**
  - `union` - union removes all duplicates.
  - Use `union all` instead of `union` to retain the duplicates.
  - `intersect`
  - `except`

Module 11

- **Various basic SQL features through example workout**
- `select distinct` - removing the duplicates
- `select all` - (duplicate retention is the default)
- Cartesian Product
- Rename AS Operation
- `where` : `AND` and `OR`
- String Operations `like` with `%` and `_`
- `order by` - `ASC` and `DESC`
- `in` Operator
- **Set Operations**
  - `union` - union removes all duplicates.
  - Use `union all` instead of `union` to retain the duplicates.
  - `intersect`
  - `except`
- **Aggregate functions** `avg`, `min`, `max`, `count`, `sum`

Module 12

- **Nested Subqueries**: A subquery is a `select-from-where` expression that is nested within another query

Module 12

- **Nested Subqueries**: A subquery is a `select-from-where` expression that is nested within another query
- **Subqueries in the where clause**

Module 12

- **Nested Subqueries**: A subquery is a `select-from-where` expression that is nested within another query
- **Subqueries in the where clause**
  - Typical use of subqueries is to perform tests:

Module 12

- **Nested Subqueries**: A subquery is a `select-from-where` expression that is nested within another query
- **Subqueries in the where clause**
  - Typical use of subqueries is to perform tests:
  - For set membership - using `in`

Module 12

- **Nested Subqueries**: A subquery is a `select-from-where` expression that is nested within another query
- **Subqueries in the where clause**
  - Typical use of subqueries is to perform tests:
  - For set membership - using `in`
  - For set comparisons - using `some`, `any`, `all`, `exists`, `not exists` and `unique`

Module 12

- **Nested Subqueries**: A subquery is a `select-from-where` expression that is nested within another query
- **Subqueries in the where clause**
  - Typical use of subqueries is to perform tests:
  - For set membership - using `in`
  - For set comparisons - using `some`, `any`, `all`, `exists`, `not exists` and `unique`
- **Subqueries in the from clause**

Module 12

- **Nested Subqueries**: A subquery is a `select-from-where` expression that is nested within another query
- **Subqueries in the where clause**
  - Typical use of subqueries is to perform tests:
  - For set membership - using `in`
  - For set comparisons - using `some`, `any`, `all`, `exists`, `not exists` and `unique`
- **Subqueries in the from clause**
  - The `with` clause provides a way of defining a temporary relation whose definition is available only to the query in which the `with` clause occurs.

Module 12

- **Nested Subqueries**: A subquery is a `select-from-where` expression that is nested within another query
- **Subqueries in the where clause**
  - Typical use of subqueries is to perform tests:
  - For set membership - using `in`
  - For set comparisons - using `some`, `any`, `all`, `exists`, `not exists` and `unique`
- **Subqueries in the from clause**
  - The `with` clause provides a way of defining a temporary relation whose definition is available only to the query in which the `with` clause occurs.
- **Subqueries in the select clause**

Module 12

- **Nested Subqueries**: A subquery is a `select-from-where` expression that is nested within another query
- **Subqueries in the where clause**
  - Typical use of subqueries is to perform tests:
  - For set membership - using `in`
  - For set comparisons - using `some`, `any`, `all`, `exists`, `not exists` and `unique`
- **Subqueries in the from clause**
  - The `with` clause provides a way of defining a temporary relation whose definition is available only to the query in which the `with` clause occurs.
- **Subqueries in the select clause**
  - Using scalar subquery - one which is used where a single value is expected.

Module 12

- **Nested Subqueries**: A subquery is a `select-from-where` expression that is nested within another query
- **Subqueries in the where clause**
  - Typical use of subqueries is to perform tests:
  - For set membership - using `in`
  - For set comparisons - using `some`, `any`, `all`, `exists`, `not exists` and `unique`
- **Subqueries in the from clause**
  - The `with` clause provides a way of defining a temporary relation whose definition is available only to the query in which the `with` clause occurs.
- **Subqueries in the select clause**
  - Using scalar subquery - one which is used where a single value is expected.
- **Modification of the Database**

Module 12

- **Nested Subqueries**: A subquery is a `select-from-where` expression that is nested within another query
- **Subqueries in the where clause**
  - Typical use of subqueries is to perform tests:
  - For set membership - using `in`
  - For set comparisons - using `some`, `any`, `all`, `exists`, `not exists` and `unique`
- **Subqueries in the from clause**
  - The `with` clause provides a way of defining a temporary relation whose definition is available only to the query in which the `with` clause occurs.
- **Subqueries in the select clause**
  - Using scalar subquery - one which is used where a single value is expected.
- **Modification of the Database**
  - Deletion of tuples from a given relation: `delete from`

Module 12

- **Nested Subqueries**: A subquery is a `select-from-where` expression that is nested within another query
- **Subqueries in the where clause**
  - Typical use of subqueries is to perform tests:
  - For set membership - using `in`
  - For set comparisons - using `some`, `any`, `all`, `exists`, `not exists` and `unique`
- **Subqueries in the from clause**
  - The `with` clause provides a way of defining a temporary relation whose definition is available only to the query in which the `with` clause occurs.
- **Subqueries in the select clause**
  - Using scalar subquery - one which is used where a single value is expected.
- **Modification of the Database**
  - Deletion of tuples from a given relation: `delete from`
  - Insertion of new tuples into a given relation: `insert into`

Module 12

- **Nested Subqueries**: A subquery is a `select-from-where` expression that is nested within another query
- **Subqueries in the where clause**
  - Typical use of subqueries is to perform tests:
  - For set membership - using `in`
  - For set comparisons - using `some`, `any`, `all`, `exists`, `not exists` and `unique`
- **Subqueries in the from clause**
  - The `with` clause provides a way of defining a temporary relation whose definition is available only to the query in which the `with` clause occurs.
- **Subqueries in the select clause**
  - Using scalar subquery - one which is used where a single value is expected.
- **Modification of the Database**
  - Deletion of tuples from a given relation: `delete from`
  - Insertion of new tuples into a given relation: `insert into`
  - Updating of values in some tuples in a given relation: `update`

Module 13

- **Join Operations**: take two relations and return as a result another relation

Module 13

- **Join Operations**: take two relations and return as a result another relation
- **Types of Join between Relations**:

Module 13

- **Join Operations**: take two relations and return as a result another relation
- **Types of Join between Relations**:
  - **Cross Join** - returns the Cartesian product of rows from tables in the join

Module 13

- **Join Operations**: take two relations and return as a result another relation
- **Types of Join between Relations**:
  - **Cross Join** - returns the Cartesian product of rows from tables in the join
  - **Inner join** - joins two table on the basis of the column which is explicitly specified in the `ON` clause with a given condition using a comparison operator

Module 13

- **Join Operations**: take two relations and return as a result another relation
- **Types of Join between Relations**:
  - **Cross Join** - returns the Cartesian product of rows from tables in the join
  - **Inner join** - joins two table on the basis of the column which is explicitly specified in the `ON` clause with a given condition using a comparison operator
  - **Equi-join** - condition containing an equality (=) operator

Module 13

- **Join Operations**: take two relations and return as a result another relation
- **Types of Join between Relations**:
  - **Cross Join** - returns the Cartesian product of rows from tables in the join
  - **Inner join** - joins two table on the basis of the column which is explicitly specified in the `ON` clause with a given condition using a comparison operator
  - **Equi-join** - condition containing an equality (=) operator
  - **Natural join** - joins two tables based on same attribute name and compatible datatypes

Module 13

- **Join Operations**: take two relations and return as a result another relation
- **Types of Join between Relations**:
  - **Cross Join** - returns the Cartesian product of rows from tables in the join
  - **Inner join** - joins two table on the basis of the column which is explicitly specified in the `ON` clause with a given condition using a comparison operator
  - **Equi-join** - condition containing an equality (=) operator
  - **Natural join** - joins two tables based on same attribute name and compatible datatypes
  - **Outer join** - computes the join and then adds tuples from one relation that does not match tuples in the other relation to the result of the join - uses null values

Module 13

- **Join Operations**: take two relations and return as a result another relation
- **Types of Join between Relations**:
  - **Cross Join** - returns the Cartesian product of rows from tables in the join
  - **Inner join** - joins two table on the basis of the column which is explicitly specified in the `ON` clause with a given condition using a comparison operator
  - **Equi-join** - condition containing an equality (=) operator
  - **Natural join** - joins two tables based on same attribute name and compatible datatypes
  - **Outer join** - computes the join and then adds tuples from one relation that does not match tuples in the other relation to the result of the join - uses null values
    - Left outer join

Module 13

- **Join Operations**: take two relations and return as a result another relation
- **Types of Join between Relations**:
  - **Cross Join** - returns the Cartesian product of rows from tables in the join
  - **Inner join** - joins two table on the basis of the column which is explicitly specified in the `ON` clause with a given condition using a comparison operator
  - **Equi-join** - condition containing an equality (=) operator
  - **Natural join** - joins two tables based on same attribute name and compatible datatypes
  - **Outer join** - computes the join and then adds tuples from one relation that does not match tuples in the other relation to the result of the join - uses null values
    - Left outer join
    - Right outer join

Module 13

- **Join Operations**: take two relations and return as a result another relation
- **Types of Join between Relations**:
  - **Cross Join** - returns the Cartesian product of rows from tables in the join
  - **Inner join** - joins two table on the basis of the column which is explicitly specified in the `ON` clause with a given condition using a comparison operator
  - **Equi-join** - condition containing an equality (=) operator
  - **Natural join** - joins two tables based on same attribute name and compatible datatypes
  - **Outer join** - computes the join and then adds tuples from one relation that does not match tuples in the other relation to the result of the join - uses null values
    - Left outer join
    - Right outer join
    - Full outer join

Module 13

- **Join Operations**: take two relations and return as a result another relation
- **Types of Join between Relations**:
  - **Cross Join** - returns the Cartesian product of rows from tables in the join
  - **Inner join** - joins two table on the basis of the column which is explicitly specified in the `ON` clause with a given condition using a comparison operator
  - **Equi-join** - condition containing an equality (=) operator
  - **Natural join** - joins two tables based on same attribute name and compatible datatypes
  - **Outer join** - computes the join and then adds tuples from one relation that does not match tuples in the other relation to the result of the join - uses null values
    - Left outer join
    - Right outer join
    - Full outer join
  - **Self-join** - a table is joined with itself

Module 13 (Cont.)

- **Views**

Module 13 (Cont.)

- **Views**
  - A view provides a mechanism to hide certain data from the view of certain users

Module 13 (Cont.)

- **Views**
  - A view provides a mechanism to hide certain data from the view of certain users
  - Any relation that is not of the conceptual model but is made visible to a user as a 'virtual relation' is called a view.

Module 13 (Cont.)

- **Views**
  - A view provides a mechanism to hide certain data from the view of certain users
  - Any relation that is not of the conceptual model but is made visible to a user as a 'virtual relation' is called a view.
  - Defining a view using the `create view` statement

Module 13 (Cont.)

- **Views**
  - A view provides a mechanism to hide certain data from the view of certain users
  - Any relation that is not of the conceptual model but is made visible to a user as a 'virtual relation' is called a view.
  - Defining a view using the `create view` statement
  - Views defined using other views

Module 13 (Cont.)

- **Views**
  - A view provides a mechanism to hide certain data from the view of certain users
  - Any relation that is not of the conceptual model but is made visible to a user as a 'virtual relation' is called a view.
  - Defining a view using the `create view` statement
  - Views defined using other views
  - **Materializing a view** - create a physical table containing all the tuples in the result of the query defining the view

Module 14

- **Integrity Constraints** - Guard against accidental damage to the database, by ensuring that authorized changes to the database do not result in a loss of data consistency

Module 14

- **Integrity Constraints** - Guard against accidental damage to the database, by ensuring that authorized changes to the database do not result in a loss of data consistency
- **Integrity Constraints on a Single Relation**

Module 14

- **Integrity Constraints** - Guard against accidental damage to the database, by ensuring that authorized changes to the database do not result in a loss of data consistency
- **Integrity Constraints on a Single Relation**
  - `not null`

Module 14

- **Integrity Constraints** - Guard against accidental damage to the database, by ensuring that authorized changes to the database do not result in a loss of data consistency
- **Integrity Constraints on a Single Relation**
  - `not null`
  - `primary key`

Module 14

- **Integrity Constraints** - Guard against accidental damage to the database, by ensuring that authorized changes to the database do not result in a loss of data consistency
- **Integrity Constraints on a Single Relation**
  - `not null`
  - `primary key`
  - `unique`

Module 14

- **Integrity Constraints** - Guard against accidental damage to the database, by ensuring that authorized changes to the database do not result in a loss of data consistency
- **Integrity Constraints on a Single Relation**
  - `not null`
  - `primary key`
  - `unique`
  - `check(P)`, where P is a predicate

Module 14

- **Integrity Constraints** - Guard against accidental damage to the database, by ensuring that authorized changes to the database do not result in a loss of data consistency
- **Integrity Constraints on a Single Relation**
  - `not null`
  - `primary key`
  - `unique`
  - `check(P)`, where P is a predicate
- **Referential Integrity** - Ensures that a value that appears in one relation for a given set of attributes also appears for a certain set of attributes in another relation

- **Integrity Constraints** - Guard against accidental damage to the database, by ensuring that authorized changes to the database do not result in a loss of data consistency
- **Integrity Constraints on a Single Relation**
  - `not null`
  - `primary key`
  - `unique`
  - `check(P)`, where P is a predicate
- **Referential Integrity** - Ensures that a value that appears in one relation for a given set of attributes also appears for a certain set of attributes in another relation
- **Cascading actions** in referential integrity:

- **Integrity Constraints** - Guard against accidental damage to the database, by ensuring that authorized changes to the database do not result in a loss of data consistency
- **Integrity Constraints on a Single Relation**
  - `not null`
  - `primary key`
  - `unique`
  - `check(P)`, where P is a predicate
- **Referential Integrity** - Ensures that a value that appears in one relation for a given set of attributes also appears for a certain set of attributes in another relation
- **Cascading actions** in referential integrity:
  - `on delete cascade`, `on update cascade`

Module 14 (Cont.)

- **Built-in Data Types in SQL** - such as `date`, `time`, `timestamp`, `interval`

Module 14 (Cont.)

- **Built-in Data Types in SQL** - such as `date`, `time`, `timestamp`, `interval`
- **Index creation** - using `create index` statement

Module 14 (Cont.)

- **Built-in Data Types in SQL** - such as `date`, `time`, `timestamp`, `interval`
- **Index creation** - using `create index` statement
- **User-Defined Types** - using `create type` statement

Module 14 (Cont.)

- **Built-in Data Types in SQL** - such as `date`, `time`, `timestamp`, `interval`
- **Index creation** - using `create index` statement
- **User-Defined Types** - using `create type` statement
- **Domains** - using `create domain` statement

Module 14 (Cont.)

- **Built-in Data Types in SQL** - such as `date`, `time`, `timestamp`, `interval`
- **Index creation** - using `create index` statement
- **User-Defined Types** - using `create type` statement
- **Domains** - using `create domain` statement
  - Types and domains are similar

Module 14 (Cont.)

- **Built-in Data Types in SQL** - such as `date`, `time`, `timestamp`, `interval`
- **Index creation** - using `create index` statement
- **User-Defined Types** - using `create type` statement
- **Domains** - using `create domain` statement
  - Types and domains are similar
  - Domains can have constraints, such as `not null`

Module 14 (Cont.)

- **Built-in Data Types in SQL** - such as `date`, `time`, `timestamp`, `interval`
- **Index creation** - using `create index` statement
- **User-Defined Types** - using `create type` statement
- **Domains** - using `create domain` statement
  - Types and domains are similar
  - Domains can have constraints, such as `not null`
- **Authorization** - using `grant` and `revoke` statement

Module 14 (Cont.)

- **Built-in Data Types in SQL** - such as `date`, `time`, `timestamp`, `interval`
- **Index creation** - using `create index` statement
- **User-Defined Types** - using `create type` statement
- **Domains** - using `create domain` statement
  - Types and domains are similar
  - Domains can have constraints, such as `not null`
- **Authorization** - using `grant` and `revoke` statement
- **Roles** - using `create role` statement

Module 15

- **SQL functions** - using `create function` statement

Module 15

- **SQL functions** - using `create function` statement
- **Table-valued functions** - can return a relation as a result

Module 15

- **SQL functions** - using `create function` statement
- **Table-valued functions** - can return a relation as a result
- **SQL procedures** - using `create procedure` statement

Module 15

- **SQL functions** - using `create function` statement
- **Table-valued functions** - can return a relation as a result
- **SQL procedures** - using `create procedure` statement
- **Language constructs for procedures and functions** - `while` loop, `repeat` loop, `for` loop, `if-then-else`, `case`

Module 15

- **SQL functions** - using `create function` statement
- **Table-valued functions** - can return a relation as a result
- **SQL procedures** - using `create procedure` statement
- **Language constructs for procedures and functions** - `while` loop, `repeat` loop, `for` loop, `if-then-else`, `case`
- **Trigger** - a set of actions that are performed in response to an `insert`, `update`, or `delete` operation on a specified table

Module 15

- **SQL functions** - using `create function` statement
- **Table-valued functions** - can return a relation as a result
- **SQL procedures** - using `create procedure` statement
- **Language constructs for procedures and functions** - `while` loop, `repeat` loop, `for` loop, `if-then-else`, `case`
- **Trigger** - a set of actions that are performed in response to an `insert`, `update`, or `delete` operation on a specified table
  - `Before` triggers and `After` triggers

Module 15

- **SQL functions** - using `create function` statement
- **Table-valued functions** - can return a relation as a result
- **SQL procedures** - using `create procedure` statement
- **Language constructs for procedures and functions** - `while` loop, `repeat` loop, `for` loop, `if-then-else`, `case`
- **Trigger** - a set of actions that are performed in response to an `insert`, `update`, or `delete` operation on a specified table
  - `Before` triggers and `After` triggers
  - `Row level` and `statement level` triggers

Module 15

- **SQL functions** - using `create function` statement
- **Table-valued functions** - can return a relation as a result
- **SQL procedures** - using `create procedure` statement
- **Language constructs for procedures and functions** - `while` loop, `repeat` loop, `for` loop, `if-then-else`, `case`
- **Trigger** - a set of actions that are performed in response to an `insert`, `update`, or `delete` operation on a specified table
  - `Before` triggers and `After` triggers
  - `Row level` and `statement level` triggers
- **Triggering events and actions in SQL**

Module 15

- **SQL functions** - using `create function` statement
- **Table-valued functions** - can return a relation as a result
- **SQL procedures** - using `create procedure` statement
- **Language constructs for procedures and functions** - `while` loop, `repeat` loop, `for` loop, `if-then-else`, `case`
- **Trigger** - a set of actions that are performed in response to an `insert`, `update`, or `delete` operation on a specified table
  - `Before` triggers and `After` triggers
  - `Row level` and `statement level` triggers
- **Triggering events and actions in SQL**
  - Triggering event can be an `insert`, `delete` or `update`

Week 4 Summary

Tuple Relational Calculus (TRC)

- TRC is a nonprocedural query language, where each query is of the form

Tuple Relational Calculus (TRC)

- TRC is a nonprocedural query language, where each query is of the form $\{t \mid P(t)\}$

Tuple Relational Calculus (TRC)

- TRC is a nonprocedural query language, where each query is of the form $\{t \mid P(t)\}$
- **where** t = resulting tuples,

Tuple Relational Calculus (TRC)

- TRC is a nonprocedural query language, where each query is of the form $\{t \mid P(t)\}$
- **where** t = resulting tuples,
- $P(t)$ = known as predicate and these are the conditions that are used to fetch t.

Tuple Relational Calculus (TRC)

- TRC is a nonprocedural query language, where each query is of the form $\{t \mid P(t)\}$
- **where** t = resulting tuples,
- $P(t)$ = known as predicate and these are the conditions that are used to fetch t.
- $P(t)$ may have various conditions logically combined with OR ($\vee$), AND ($\wedge$), NOT($\neg$).

Tuple Relational Calculus (TRC)

- TRC is a nonprocedural query language, where each query is of the form $\{t \mid P(t)\}$
- **where** t = resulting tuples,
- $P(t)$ = known as predicate and these are the conditions that are used to fetch t.
- $P(t)$ may have various conditions logically combined with OR ($\vee$), AND ($\wedge$), NOT($\neg$).
- It also uses **quantifiers**:

Tuple Relational Calculus (TRC)

- TRC is a nonprocedural query language, where each query is of the form $\{t \mid P(t)\}$
- **where** t = resulting tuples,
- $P(t)$ = known as predicate and these are the conditions that are used to fetch t.
- $P(t)$ may have various conditions logically combined with OR ($\vee$), AND ($\wedge$), NOT($\neg$).
- It also uses **quantifiers**:
  - $\exists\ t \in r\ (Q(t))$ = 'there exists' a tuple in t in relation r such that predicate $Q(t)$ is true.

Tuple Relational Calculus (TRC)

- TRC is a nonprocedural query language, where each query is of the form $\{t \mid P(t)\}$
- **where** t = resulting tuples,
- $P(t)$ = known as predicate and these are the conditions that are used to fetch t.
- $P(t)$ may have various conditions logically combined with OR ($\vee$), AND ($\wedge$), NOT($\neg$).
- It also uses **quantifiers**:
  - $\exists\ t \in r\ (Q(t))$ = 'there exists' a tuple in t in relation r such that predicate $Q(t)$ is true.
  - $\forall\ t \in r\ (Q(t))$ = $Q(t)$ is true 'for all' tuples in relation r.

Tuple Relational Calculus (TRC)

- TRC is a nonprocedural query language, where each query is of the form $\{t \mid P(t)\}$
- **where** t = resulting tuples,
- $P(t)$ = known as predicate and these are the conditions that are used to fetch t.
- $P(t)$ may have various conditions logically combined with OR ($\vee$), AND ($\wedge$), NOT($\neg$).
- It also uses **quantifiers**:
  - $\exists\ t \in r\ (Q(t))$ = 'there exists' a tuple in t in relation r such that predicate $Q(t)$ is true.
  - $\forall\ t \in r\ (Q(t))$ = $Q(t)$ is true 'for all' tuples in relation r.
- Example: $\{P \mid \exists\ S \in \text{Students}\ (S.\text{CGPA} > 8 \wedge P.\text{name} = S.\text{sname} \wedge P.\text{age} = S.\text{age})\}$

Tuple Relational Calculus (TRC)

- TRC is a nonprocedural query language, where each query is of the form $\{t \mid P(t)\}$
- **where** t = resulting tuples,
- $P(t)$ = known as predicate and these are the conditions that are used to fetch t.
- $P(t)$ may have various conditions logically combined with OR ($\vee$), AND ($\wedge$), NOT($\neg$).
- It also uses **quantifiers**:
  - $\exists\ t \in r\ (Q(t))$ = 'there exists' a tuple in t in relation r such that predicate $Q(t)$ is true.
  - $\forall\ t \in r\ (Q(t))$ = $Q(t)$ is true 'for all' tuples in relation r.
- Example: $\{P \mid \exists\ S \in \text{Students}\ (S.\text{CGPA} > 8 \wedge P.\text{name} = S.\text{sname} \wedge P.\text{age} = S.\text{age})\}$
  - Returns the name and age of students with a CGPA above 8.

Domain Relational Calculus (DRC)

- $\{\langle x_1, x_2, \ldots, x_n \rangle \mid P(x_1, x_2, \ldots, x_n)\}$

Domain Relational Calculus (DRC)

- $\{\langle x_1, x_2, \ldots, x_n \rangle \mid P(x_1, x_2, \ldots, x_n)\}$
- $x_1, x_2, \ldots, x_n$ represent domain variables

Domain Relational Calculus (DRC)

- $\{\langle x_1, x_2, \ldots, x_n \rangle \mid P(x_1, x_2, \ldots, x_n)\}$
- $x_1, x_2, \ldots, x_n$ represent domain variables
- P represents a formula similar to that of the predicate calculus

Domain Relational Calculus (DRC)

- $\{\langle x_1, x_2, \ldots, x_n \rangle \mid P(x_1, x_2, \ldots, x_n)\}$
- $x_1, x_2, \ldots, x_n$ represent domain variables
- P represents a formula similar to that of the predicate calculus
- Example Relation Students (Name, Age, Marks, Subject)

Domain Relational Calculus (DRC)

- $\{\langle x_1, x_2, \ldots, x_n \rangle \mid P(x_1, x_2, \ldots, x_n)\}$
- $x_1, x_2, \ldots, x_n$ represent domain variables
- P represents a formula similar to that of the predicate calculus
- Example Relation Students (Name, Age, Marks, Subject)
- Example Query: $\{\langle a, b \rangle \mid \exists\ a, b, c, d(\langle a, b, c, d \rangle \in \text{students} \wedge c > 75)\}$

Domain Relational Calculus (DRC)

- $\{\langle x_1, x_2, \ldots, x_n \rangle \mid P(x_1, x_2, \ldots, x_n)\}$
- $x_1, x_2, \ldots, x_n$ represent domain variables
- P represents a formula similar to that of the predicate calculus
- Example Relation Students (Name, Age, Marks, Subject)
- Example Query: $\{\langle a, b \rangle \mid \exists\ a, b, c, d(\langle a, b, c, d \rangle \in \text{students} \wedge c > 75)\}$
  - Returns the name and age of students having marks above 75.

Domain Relational Calculus (DRC)

- $\{\langle x_1, x_2, \ldots, x_n \rangle \mid P(x_1, x_2, \ldots, x_n)\}$
- $x_1, x_2, \ldots, x_n$ represent domain variables
- P represents a formula similar to that of the predicate calculus
- Example Relation Students (Name, Age, Marks, Subject)
- Example Query: $\{\langle a, b \rangle \mid \exists\ a, b, c, d(\langle a, b, c, d \rangle \in \text{students} \wedge c > 75)\}$
  - Returns the name and age of students having marks above 75.
- **Note**: You have to mention the domain variables in the same order as given in the table.

Example Problem Q1

Relation Students

Name	Age	Marks	Subject
David	23	78	Maths
Matthew	29	54	English
Anand	22	89	JAVA
Mitchel	21	56	Maths
Shaun	26	92	Maths

Relation Activity

Name	Sports	Awards	Points
David	Cricket	2	67
Matthew	Football	4	90
Anand	Cricket	5	80
Mitchel	Tennis	8	70
Shaun	Hockey	3	75

Example Problem Q1

Relation Students

Name	Age	Marks	Subject
David	23	78	Maths
Matthew	29	54	English
Anand	22	89	JAVA
Mitchel	21	56	Maths
Shaun	26	92	Maths

Relation Activity

Name	Sports	Awards	Points
David	Cricket	2	67
Matthew	Football	4	90
Anand	Cricket	5	80
Mitchel	Tennis	8	70
Shaun	Hockey	3	75

Q). Return the names of students whose age is greater than 25, or who are enrolled in Maths.

- **RA**: $\Pi_{\text{Name}}(\sigma_{\text{age}>25 \vee \text{subject}=\text{'Maths'}}(\text{Students}))$
- **TRC**: $\{t \mid \exists\ s \in \text{students}(s.\text{age} > 25 \vee s.\text{subject} = \text{'Maths'} \wedge t.\text{name} = s.\text{name})\}$

Example Problem Q2

Relation Students

Name	Age	Marks	Subject
David	23	78	Maths
Matthew	29	54	English
...	...	...	...

Relation Activity

Name	Sports	Awards	Points
David	Cricket	2	67
Matthew	Football	4	90
...	...	...	...

Example Problem Q2

Relation Students

Name	Age	Marks	Subject
David	23	78	Maths
Matthew	29	54	English
...	...	...	...

Relation Activity

Name	Sports	Awards	Points
David	Cricket	2	67
Matthew	Football	4	90
...	...	...	...

Q). Return the names of students along with sports, whose age is less than 25 and who have secured more than 75 marks.

- **RA**: $\Pi_{\text{Name,Sports}}(\sigma_{\text{age}<25 \wedge \text{Marks}>75}(\text{Students} \bowtie \text{Activity}))$
- **TRC**: $\{t \mid \exists\ s \in \text{students} \exists\ a \in \text{activity}(s.\text{name} = a.\text{name} \wedge s.\text{age} < 25 \wedge s.\text{marks} > 75 \wedge t.\text{name} = s.\text{name} \wedge t.\text{sports} = a.\text{sports})\}$
- **DRC**: $\{\langle a, f\rangle \mid \exists\ b, c, d(\langle a, b, c, d\rangle \in \text{students} \wedge b < 25 \wedge c > 75) \wedge \exists\ e, g, h(\langle e, f, g, h\rangle \in$

E-R Diagram and Mapping Constraints

- Please go through the tutorials.

E-R Diagram and Mapping Constraints

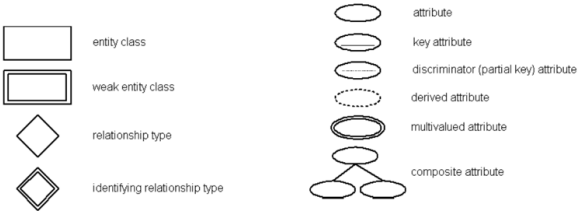
- Please go through the tutorials.

many-to-many relationship

one-to-many relationship

one-to-one relationship

many-to-one relationship

Total and Partial Participation

E-R diagrams symbols